

Adventure Game Project Report

Team 2: Owen Rooff, Ty Glatt, Ryan Heiser, Joseph Moore

CPSC 224 – Software Development, Fall 2025

12/12/2025

Abstract:

Adventure Game is a turn based fighting game with rogue-like elements. The game includes the exploration of a procedurally generated, grid-based level. The rooms in the level each have a chance of spawning an enemy creature and an item. In addition to these rooms, there is also one shop and one boss room in the level. Entering a room with an enemy or the boss starts a fight, where the user can attack the enemy with weapons as well as use consumables, accessible from the inventory. We initially intended to have multiple levels per run, with increasing difficulty, but we did not have the time to implement it.

Introduction and Requirements

For our project we decided to create a singleplayer Rogue-like game that takes inspiration from games such as Binding of Issac and old Pokémon classics. Furthermore, our game has a procedurally generated level with each level containing their own rooms, boss and shop. Lastly, it contains its own damage type system, and consumables for the player to utilize.

At the start of the game, it gives the player an option of 4 classes to choose from, and each class contains its own two starting weapons and consumables. Our game also contains a basic level design/exploration that heavily derives from Binding of Issac's level design and style of exploration with there being several rooms in each level for the player to explore that contain mobs and items for the player to fight/pick up. Each level also contains a boss room which needs to be defeated in order to progress to other levels, and also a shop room for the player to spend gold, and get better weapons/consumables. For our fighting system, we took light inspiration from Pokémon games in terms of their turn-based fighting system where the player and the mob take turns to use weapons and consumables until one or the other are defeated. Furthermore, our fighting system also contains both a dodge/evasion system and a damage type system. For the dodge system it takes into account the accuracy percentage of the chosen weapon and also the evasion stat of the mob that the player is currently fighting against to determine the mob's dodge chance. For the damage type system, each class, and mob has certain strengths/weaknesses to damage types which either provide resistance or a vulnerability to certain attacks(e.g the troll mob is weak towards magic and slice damage while being resistant to blunt and projectile damage). Moreover, each weapon contains a single move that either does blunt, slice, projectile, or magic damage based on the type of move it is(e.g the steel sword weapon has a slice move). To conclude, we have a basic consumable system that gives the player certain boons based on what it's for.

Defined user stories

#	Title	Story	Priority level	Completed or not
1	Beat the game	As a completionist, I can win the game so that I feel a sense of accomplishment.	High	Completed
2	Buy items	As a gamer, I can buy items from a shop so I can make my character stronger.	High	Completed
3	Exploration	As a player, I can wander through the level so that I can fight enemies, find items, and progress.	High	Completed
4	Ability to use items	As a player in a battle, I can select an item to use so that its benefits are given to my character.	Medium	Completed
5	Upgrade character	As a Rogue-like game, I can upgrade my character as I progress so that I can progress further and enjoy my gaming experience.	High	Completed
6	Choose an attack	As a player in a battle, I can select a move to use so that it attacks an opponent.	High	Completed
7	Ability to fight mobs	As a turn-based fighting gamer, I can be attacked by computer enemies so that I have a challenge in my game.	High	Completed
8	Save game	As a busy gamer, I can save the game to come back to later so that I	low	Incomplete

		can continue my run when I have time.		
9	Reach checkpoints	As a player, I can reach checkpoints so that I can progress through the game.	medium	Incomplete

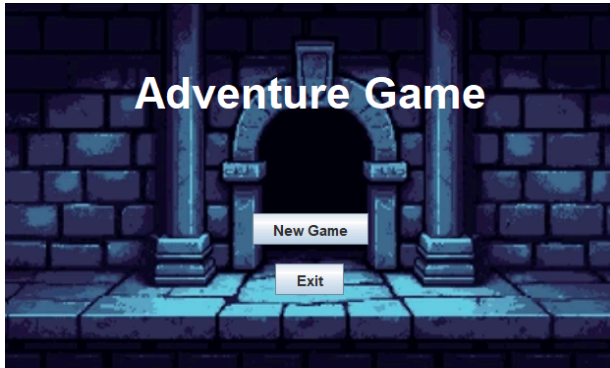
Acceptance criterias

#	Story title	Acceptance criteria
1	Beat the game	Final boss fight, win screen, keep track of number of wins. Must beat the final boss, must be in a run.
2	Buy items	Player can collect money, shop generates on the map, player can use money to buy an item. Must have enough money, must be in the shop.
3	Exploration	Character in level, walk around, interact with environment.
4	Ability to use items	It is your turn, cycle to items, select items to use, item is used, item gives benefits to player, turn ends.
5	Upgrade character	Find and use items, unlock perks, view items/perks that have been collected. Must be in the middle of a run, must explore to find items/perks.
6	Choose an attack	Is your turn, cycle to move, select move, move reduces opponents health, move inflicts effects, turn ends.
7	Ability to fight mobs	Computer chooses moves, computer inflicts damage, turn ends after move is used. Both characters must be alive, the player must come across an enemy.

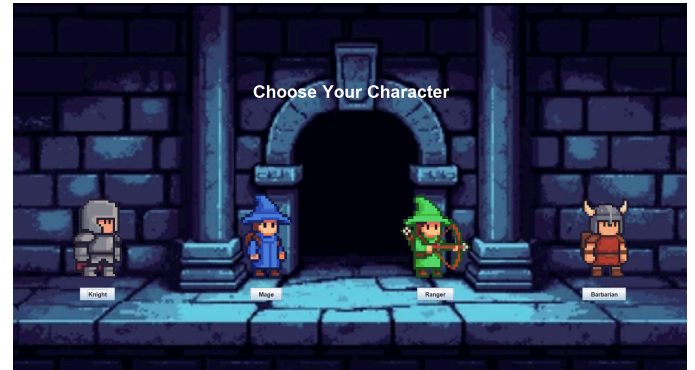
Software Implementation

This walk-through shows what the player would see in chronological order of a typical run of Adventure Game.

When the application is first run, the user is met with the title screen, where the user can either start a new game or exit the application. Once they click the 'New Game' button, they are then met with a character selection screen, where they can choose one of four characters to use.

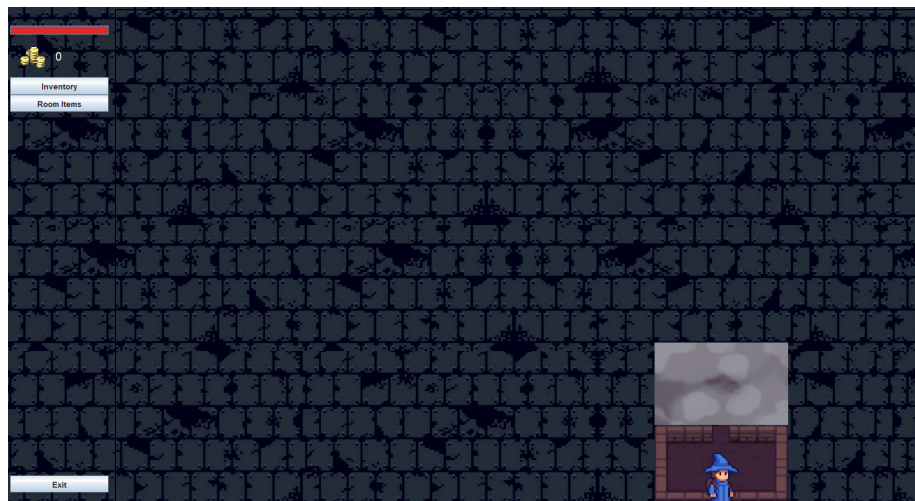


Start Screen

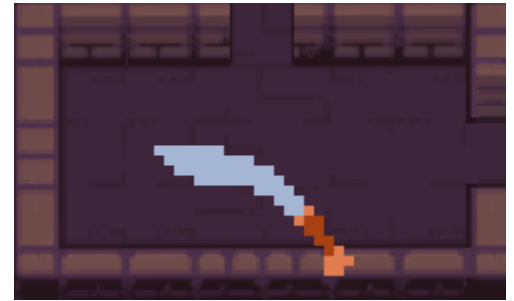


Character Select Screen

After choosing a character, they are then shown the level. The level is procedurally generated, meaning that each time the game is run, a completely different level is created through a set of rules. The sprite of the chosen character will be on the current tile of the player. The user can move to any available adjacent tile by clicking on it. Unexplored tiles have a fog texture, which shows where they haven't gone yet. The user can also see their health & gold in the top left, as well as access to their inventory and any items that are in the room.



While moving through the level, the user may encounter what looks like a weapon on the ground. This indicates an item in the room, which they can pick up. So far, we have mainly seen elements of the game that help make it a rogue-like, such as the character selection and procedural generation. When the player goes to the next tile however, we get into our first fight against an enemy.



The visual layout of the fighting system is a mix between Slay the Spire and Pokémon. We wanted to be able to see the faces of both the player and the enemy while still looking at each other, which was inspired by Slay the Spire. We also wanted to have a similar weapon layout to Pokémon, where your four possible attacks are in a grid.

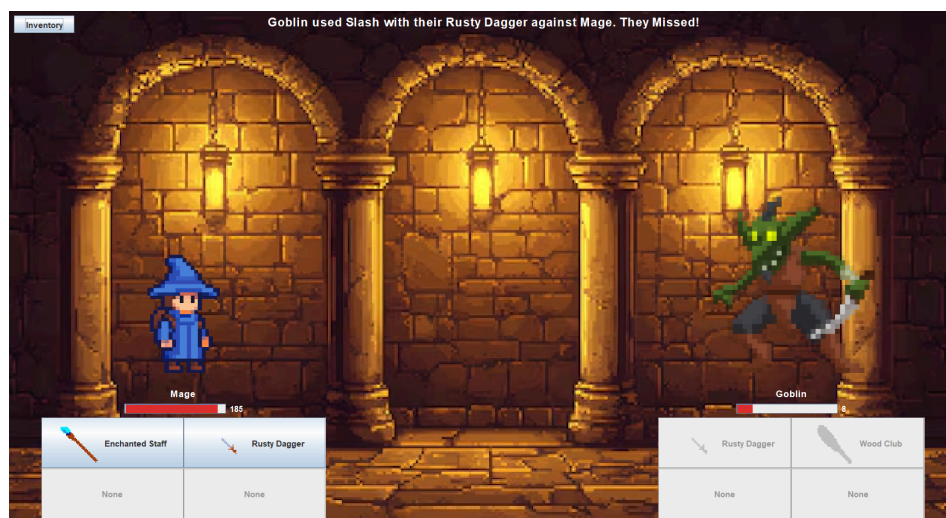


Slay the Spire

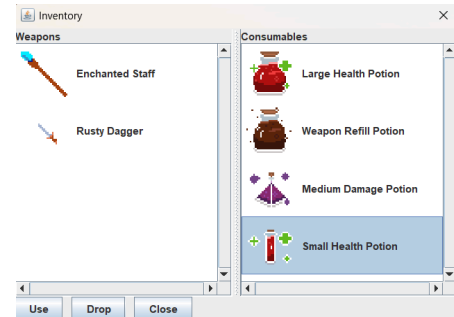


Pokémon

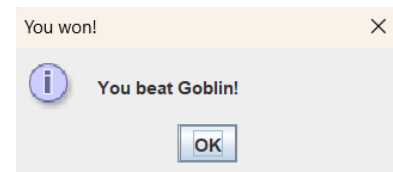
The user can select a weapon by clicking the button. If they hover the mouse over a weapon, its stats will show up, including tier, damage, accuracy, and uses left (if applicable). At the top of the screen, text is displayed to show what the latest move was. It shows what weapon was chosen, as well how and who it affected.



They can also access the inventory in the top left, where they can see their weapons and consumables, as well as actually use the consumables. The weapons and consumables also have sprites to make them visually appealing.



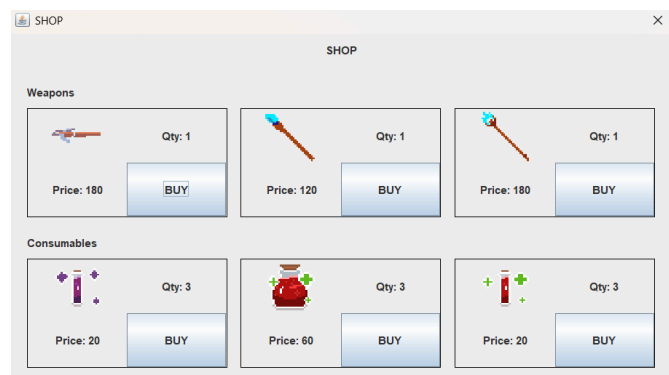
The player and the enemy take turns attacking one another until either the player or the enemy dies. If the player perishes, they get a notification saying they died, then get kicked back to the title screen. This idea of resetting after death is a core part of rogue-like games. If the enemy is killed, then the user gets a pop-up saying the enemy got defeated, which then takes them back to the level. The player also gets gold for defeating the enemy.



While traversing the level, the user will most likely see a room with a bag. This represents the shop, and when clicked on pulls up the shop menu, where they can buy weapons and consumables with the gold obtained from beating enemies.



Shop Room Icon



Shop Menu

The user will soon come across an unexplored room with a mysterious sprite. This indicates that this room is the boss room. The user should be sure that they have good enough weapons and consumables before entering. Once the player decides to take on the boss, there is no going back. The player is then faced off against one of three powerful foes.



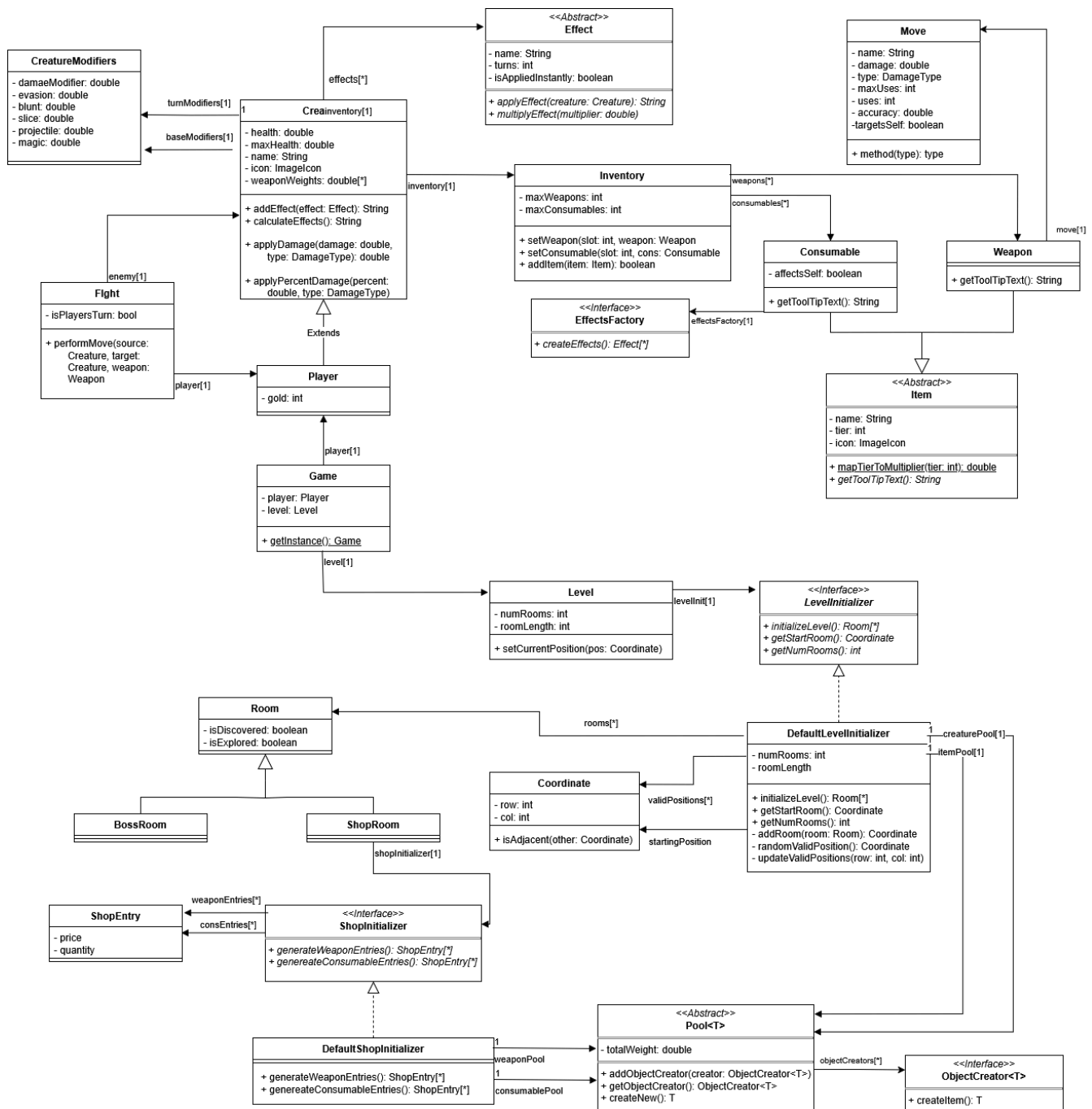
Once the player successfully defeats the boss, they are met with the victory screen.



Software Design

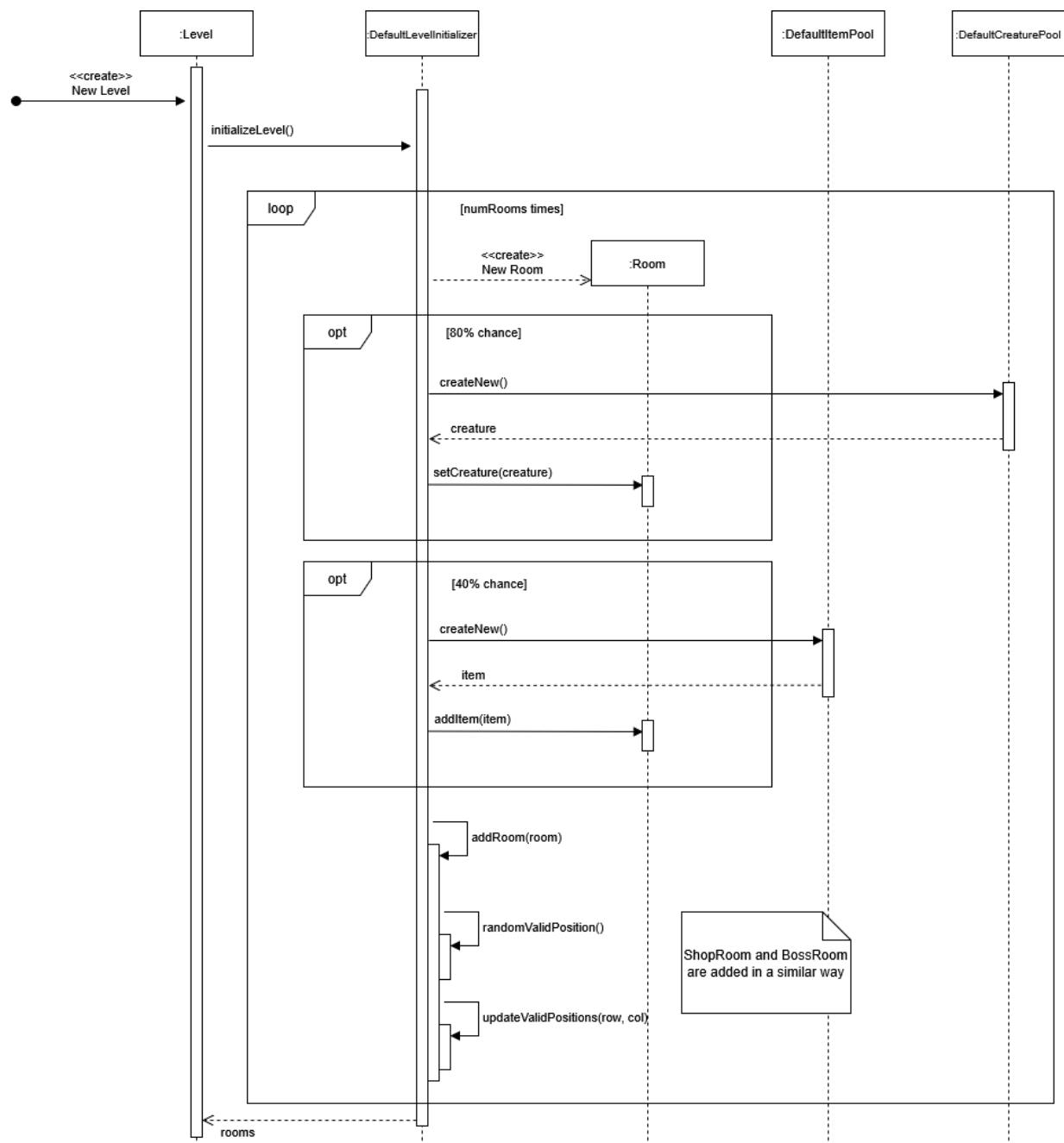
Class Diagram

The Game class is a Singleton. Implementations of the LevelInitializer interface set up Rooms for a Level in a 2D grid. Shops are initialized by an implementation of the ShopInitializer interface. Shops consist of ShopEntries which represent individual Items that can be bought. Creatures have an Inventory, Effects, and Modifiers. Inventory consists of Weapons and Consumables. Weapons have a Move which determines how an attack is performed. Consumables have effects which are added to a Creature on use. The Effects of a Creature have a set number of turns and are applied each turn until they run out. Implementations of the Effects abstract class determines what the Effect does.



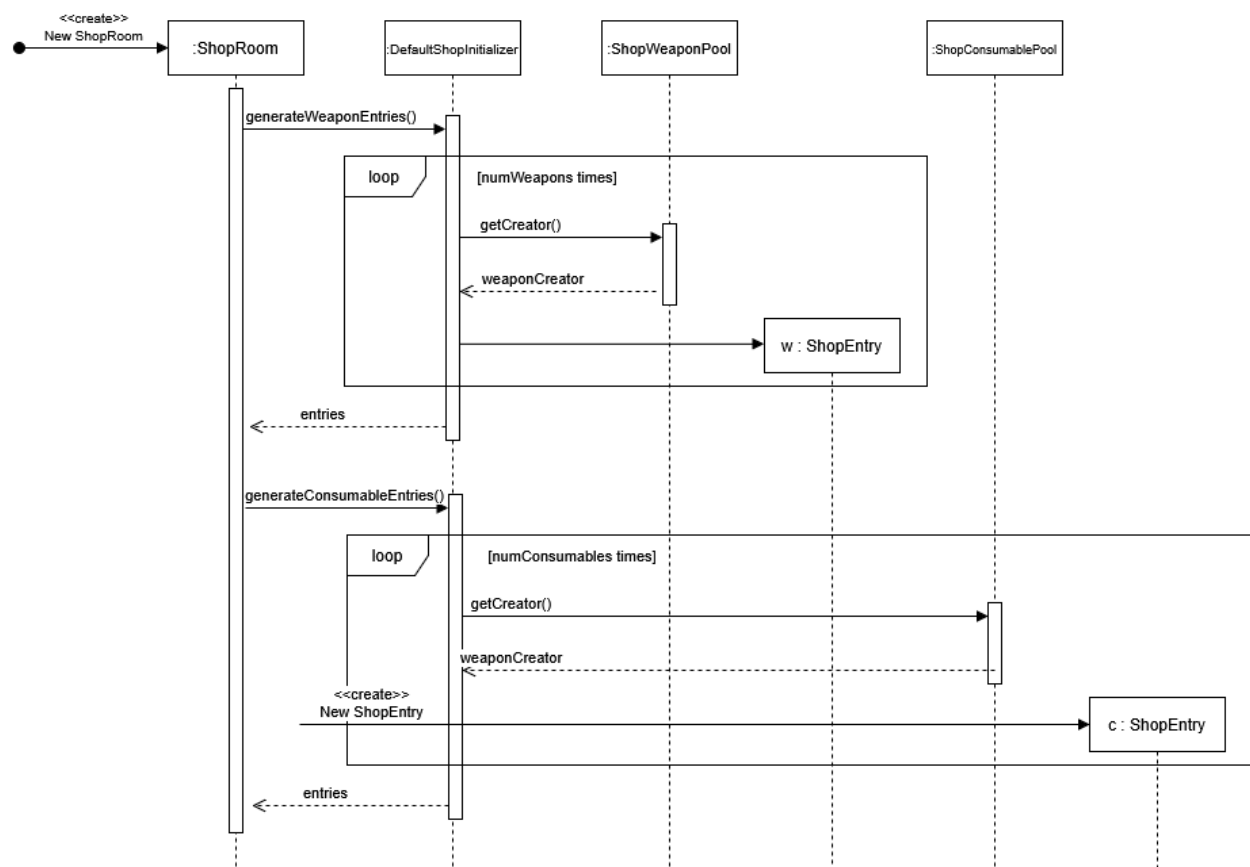
Level Initialization Sequence Diagram

When a new Level is created, it delegates the creation and layout of its Rooms to its LevelInitializer. In this case, the LevelInitializer is an instance of a DefaultLevelInitializer. The number of Rooms created is specified in the constructor of the initializer. These Rooms are laid out in a square with a length also specified in the constructor. Each Room has an 80% chance to add a Creature from the DefaultCreaturePool. There is a 40% chance for a Room to add an Item from the DefaultItemPool. These pools extend the Pool<T> abstract class, which consists of many ObjectCreator<T> functional interfaces. Each of these ObjectCreators is assigned a weight when added to the Pool class which determines the likelihood of being selected. After a Room is initialized, it is added to a valid position. Valid positions are stored in a HashSet and updated each time a Room is added.



Shop Initialization Sequence Diagram

When a new Shop is created, it delegates the creation of its ShopEntries to its ShopInitializer. This diagram shows the initializer as an instance of a DefaultShopInitializer. The number of Weapons and Consumables as well as the quantity for each ShopEntry is specified in the DefaultShopInitializer's constructor. Each weapon ShopEntry is given an ObjectCreator<Weapon> on initialization. Similarly, each consumable ShopEntry is given an ObjectCreator<Consumable>. When the ShopEntry is bought from, it creates a new Item each time rather than giving the player the same instance of the Item for each purchase.



Testing

We wrote 129 total unit tests. Each concrete class has 100% method coverage. This is not including classes containing GUI elements or our factory classes which consist only of static methods to create creatures, items, moves, etc. To compensate for our lack of automated GUI tests, we made sure to manually test each feature we added or changed to ensure it was working properly.

All Tests:

✓ 129 tests passed 129 tests total, 1 sec 167 ms

Test Coverage:

Element	Class, % ▾	Method, %	Line, %	Branch, %
✓ 📁 cpsc224	100% (31/31)	100% (178/178)	97% (527/539)	89% (140/156)
> 📁 effects	100% (7/7)	100% (30/30)	100% (53/53)	100% (4/4)
> 📁 levels	100% (7/7)	100% (46/46)	98% (135/137)	95% (63/66)
> 📁 pools	100% (6/6)	100% (9/9)	97% (68/70)	66% (4/6)
> 📁 items	100% (4/4)	100% (39/39)	100% (116/116)	93% (30/32)
> 📁 creatures	100% (4/4)	100% (44/44)	95% (116/121)	86% (26/30)
🕒 Fight	100% (1/1)	100% (3/3)	89% (26/29)	58% (7/12)
🕒 Game	100% (1/1)	100% (6/6)	100% (8/8)	100% (2/2)
> 📁 damagetypes	100% (1/1)	100% (1/1)	100% (5/5)	100% (4/4)

Project Statistics

Files: 132

Images: 38 (including 3 sprite sheets)

Tests: 129

Lines of code: 7076

Contribution using git-quick-stats

Joseph Moore

Insertions: 1123

Deletions: 298

Lines changed: 1421

Owen Roof

Insertions: 595

Deletions: 62

Lines changed: 657

Ryan Heiser

Insertions: 8359

Deletions: 3095

Lines changed: 11,454

Ty Glatt Lines

Insertions: 602

Deletions: 41

Lines changed: 643

Summary and Future Work

Throughout the process of creating our game our team learned many lessons. We consistently underestimated the amount of time it would take to implement functions. We found that bug fixing and problem solving also took considerable amounts of time. You think you've implemented something correctly just to run it, find out it doesn't work, fix it and then repeat the process. The creation of textures was also difficult. Our team used a combination of assets found online as well as assets we created. Getting the GUI elements to format and look the desired way was yet another challenge. If we were to take different steps a smaller original scope would be helpful. Planning out exactly how systems/functions should work before coding would have saved a lot of time and energy as well. While our team did face challenges a lot of things went well. The implementation of the fighting system as well as exploration system are particularly good. Future possible implementations include: more levels, more items, more enemies, and game balance changes.